



COURSE MATERIAL

Making Decisions

CMSC 131 Bootcamp Block 5

Prepared by: Rene Andre Bedonia Jocsing

Published: September 09, 2026

Deadline: September 09, 2026

Prerequisites

One archive holds everything this block needs, including a project that already builds. You can start today without Git, without a GitHub account, and without last session's folder.

You need this [archive](#). Unzip it somewhere permanent and work inside the folder it makes.

Session Objectives

- Explain what `cmp` changes and what it leaves alone
- Choose the correct conditional jump for signed and for unsigned comparisons
- Translate `if`, `if/else`, `while`, and `for` into labels and branches
- Write a loop that terminates, and say why it does

Scoring

This block is worth 10 points for work completed during its scheduled laboratory session. Your instructor checks your progress before the session ends and prorates the 10 points according to how much of the block you completed. Complete all seven guided blocks, `b1` through `b7`, without an absence and you earn a 30-point completion bonus. Attendance is checked during every bootcamp session, so it doesn't carry a separate score.

Before You Start

The archive is a working project with today's files already in it. If you prefer to keep working in your own folder, copy these across instead.

File	What it's for
<code>b5_starter.asm</code>	Today's exercise, with the prompts written and the branching left to you
<code>classify.input</code>	One number, fed to your program by <code>make check</code>
<code>classify.expected</code>	What a correct program prints for it
<code>b4_solution.asm</code>	Block 4's program with its bug fixed, if you missed that session

Ninety minutes, roughly fifteen on the flags, twenty on the jump families, twenty on translating the control structures you already know, and thirty-five on the exercise.

Part 1: The Flags Register

Assembly has no `if`. It has `jmp`, which moves execution to a label, and a family of conditional jumps that move execution only when some condition holds.

The condition is never written into the jump. It's read from the **flags register**, a set of single-bit results left behind by the previous arithmetic instruction. The three that matter here are:

Flag	Set when
ZF, the zero flag	The result was exactly zero
SF, the sign flag	The result's top bit was 1, meaning negative if read as signed
CF, the carry flag	The operation carried or borrowed past the end, meaning out of range if read as unsigned

`cmp` is how you set them:

```
1      cmp     eax, ebx
```

`cmp` performs `eax - ebx`, sets the flags from the result, and **throws the result away**. `eax` is unchanged. It exists purely to leave a trace in the flags for the jump that follows.

So `cmp eax, ebx` followed by `je` somewhere reads as “jump if `eax - ebx` was zero”, which is another way of saying “jump if they were equal”.

Part 2: The Conditional Jumps

Each jump asks about the flags. The names read as “jump if ...”.

For signed comparisons, used for ordinary numbers that may be negative:

Instruction	Jumps when	Reads as
<code>je</code>	equal	<code>==</code>
<code>jne</code>	not equal	<code>!=</code>
<code>jl</code>	less	<code><</code>
<code>jle</code>	less or equal	<code><=</code>
<code>jg</code>	greater	<code>></code>
<code>jge</code>	greater or equal	<code>>=</code>

For unsigned comparisons, used for addresses, sizes, and counts that can't be negative:

Instruction	Jumps when	Reads as
<code>jb</code>	below	<code><</code>
<code>jbe</code>	below or equal	<code><=</code>
<code>ja</code>	above	<code>></code>
<code>jae</code>	above or equal	<code>>=</code>

`je` and `jne` serve both. Equality doesn't care about sign.

Why two families? Block 3 established that a bit pattern carries no sign. The same is true here. `cmp` sets the flags, but nothing in the flags says which reading you meant, so the jump has to.

Compare `-1` against `1`:

- Signed, $-1 < 1$, so `j1` takes the branch.
- Unsigned, -1 is 4,294,967,295, which is well above 1, so `jb` doesn't.

Same two registers, same `cmp`, opposite outcomes. Picking the wrong family gives you a program that's correct for small positive numbers and wrong everywhere else, which is a bug that survives casual testing.

Use the signed family for values a user typed. Use the unsigned family for counts, lengths, and addresses.

Part 3: Translating What You Know

if

```
1  if (x > 10) {
2      y = 1;
3  }
```

The trick is to invert the condition. In C you state the case you want. In assembly you jump *past* the body when you don't want it.

```
1      cmp     eax, 10
2      jle     skip_body      ; NOT greater, so skip
3      mov     ebx, 1
4  skip_body:
```

if / else

```
1  if (x > 10) { y = 1; } else { y = 2; }
```

```
1      cmp     eax, 10
2      jle     else_part
3      mov     ebx, 1
4      jmp     end_if          ; do not fall into the else
5  else_part:
6      mov     ebx, 2
7  end_if:
```

Without that `jmp end_if`, the true branch runs and then keeps going straight into the false branch, so `ebx` ends up 2 no matter what. Every `if/else` needs that unconditional jump. Forgetting it is the most common control-flow mistake in this course.

while

```
1  while (x > 0) { x = x - 1; }
```

```
1  while_top:
2      cmp     eax, 0
3      jle     while_end      ; test at the top
4      dec     eax
5      jmp     while_top
6  while_end:
```

for

```
1  for (i = 0; i < 10; i++) { ... }
```

```
1      mov     ecx, 0
2  for_top:
3      cmp     ecx, 10
4      jge     for_end
5      ; body here
6      inc     ecx
7      jmp     for_top
8  for_end:
```

That for loop needs a `cmp` and two jumps per iteration. Counting down is cheaper. `dec` sets the zero flag by itself:

```
1      mov     ecx, 10
2  count_down:
3      ; body here
4      dec     ecx                ; sets ZF when ecx reaches 0
5      jnz     count_down        ; no cmp needed
```

The count-down version costs one instruction less per iteration and needs no separate comparison. This is why so much hand-written assembly counts down when the order of iterations doesn't matter.

Be careful when it does matter. If the body uses `ecx` as an index, counting down visits elements in reverse, which is fine for summing and wrong for printing.

Part 4: Printing One Character

The exercise below wants 1 2 3 4 5 on a line, which means printing a space between numbers. Every routine you've used so far prints either a whole string or a whole integer.

`print_char` is the one you want. It prints the single character whose code is in `eax`. NASM will work out that code for you if you write the character in quotes:

```
1      mov     eax, ' '
2      call    print_char
```

Declaring `space db " ", 0` in `.data` and calling `print_string` also works and costs you a label. Either is fine. `print_char` is the one worth remembering. Block 6 needs it again for a different reason.

Where you print the space matters more than how. Printing one after every number leaves a space at the end of the line, dangling after the 5. You can't see it, your terminal won't show it, and `check` compares bytes without reading them for meaning, so it fails and shows you two lines that appear identical. Print the separator *before* each number except the first and the problem doesn't arise.

Part 5: Exercise

Start from `b5_starter.asm`, which has the six messages declared and the branching left to you.

```
1  cp b5_starter.asm classify.asm
```

Write a program that reads one integer and classifies it, then counts.

1. Read an integer.
2. Print `negative`, `zero`, or `positive` using `cmp` and conditional jumps.
3. If it's positive, print every integer from 1 up to it, on one line, separated by spaces.
4. Then print the sum of those integers.
5. If it isn't positive, skip steps 3 and 4 and say so.

Expected output:

```

1 Enter an integer: 5
2 positive
3 1 2 3 4 5
4 sum: 15
5
6 Enter an integer: -3
7 negative
8 nothing to count
9
10 Enter an integer: 0
11 zero
12 nothing to count

```

Constraints. Use the signed jump family, since the input can be negative. Write the counting loop twice, once counting up and once counting down, and keep whichever one prints in the right order.

Counting down is one instruction cheaper per iteration, but this exercise needs ascending output. Which loop did you keep, and what did that cost you?

Checking it

```
1 make PROG=classify check
```

`classify.input` holds 5, so that's the case being checked. Run the other three by hand. A program that gets 5 right and loops forever on 0 passes this check and is still broken. That's why the checklist below lists more cases than the fixture covers.

Testing Checklist

Core Functionality

- 5 prints `positive`, then 1 2 3 4 5, then `sum: 15`
- -3 prints `negative` and skips the counting
- 0 prints `zero` and skips the counting
- 1 prints `positive`, then 1, then `sum: 1`
- No input causes an infinite loop
- `make PROG=classify check` prints OK: `classify` matches `classify.expected`

Common Pitfalls

- A trailing space after the last number, which is invisible on screen and fails check
- Omitting the `jmp` at the end of a true branch, so both branches run
- Using `jb/ja` on user input, which misreads every negative number as huge
- Comparing in the wrong order, since `cmp eax, ebx` then `j1` means `eax < ebx`
- A loop whose counter is modified inside the body, so the exit test never holds
- Placing a label inside the loop body by accident, which turns two loops into one

Architecture Review

What We Built

- Branch-based versions of all four control structures you already knew
- A loop that provably terminates, and a second one that's cheaper
- Correct handling of negative input through the signed jump family

Key Takeaways

1. **cmp subtracts and discards.** Its only product is the flags.
2. **Assembly inverts your conditions.** You jump past the body you don't want.
3. **Signed and unsigned jumps are different instructions.** The flags can't tell them apart for you.
4. **Every if/else needs a trailing unconditional jump.** Otherwise both halves run.
5. **dec plus jnz beats cmp plus jge.** Take it when iteration order is free.

Next Session Preview

- Money, and why floating point is the wrong tool for it
- Storing pesos as whole centavos
- A loop that carries a running balance across iterations
- Printing a fixed-point value so it reads like currency